

# GUIA DE COMANDOS

## Python, Pytest, Playwright, Selenium e Allure

Ambiente virtual, plugins, execução dos testes e relatórios

Material organizado para Rodrigo Costa

**Objetivo:** Deixar os comandos em uma sequência prática: criar a máquina virtual, instalar dependências, configurar o pytest, executar o projeto e gerar relatórios Allure.

### Resumo rápido - sequência recomendada

**Uso principal:** Use esta sequência quando estiver configurando o projeto pela primeira vez em uma máquina ou após remover o ambiente virtual.

```
cd "C:\Users\RODRIGO COSTA\Desktop\Automação\PlayWright\Automacao"
python --version
py -0p
python -m venv mv
.\mv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
python -m pip install pytest-playwright playwright allure-pytest pytest-order pytest-base-url
selenium
playwright install
python -m pip freeze > requirements.txt
python -m pytest
```

## 1. Pré-requisitos

- Instalar o Python no Windows e marcar a opção Add Python to PATH durante a instalação.
- Instalar o PyCharm, caso ele seja sua IDE principal.
- Ter o projeto aberto na pasta raiz, onde ficam pytest.ini, tests, pages e requirements.txt.
- Usar o PowerShell ou CMD conforme o comando de ativação do ambiente virtual.

| Ordem | Comando                    | Para que serve                                                              |
|-------|----------------------------|-----------------------------------------------------------------------------|
| 1     | python --version           | Confere se o Python está instalado e mostra a versão ativa no terminal.     |
| 2     | py -0p                     | Lista as versões do Python instaladas na máquina e seus caminhos.           |
| 3     | cd "C:\caminho\do\projeto" | Entra na pasta raiz do projeto antes de criar ou ativar o ambiente virtual. |

## 2. Criar, ativar e remover a máquina virtual

**Nome usado:** Neste guia o ambiente virtual se chama mv, seguindo o padrão que você já está usando no projeto.

| Ordem | Comando                        | Para que serve                                                                               |
|-------|--------------------------------|----------------------------------------------------------------------------------------------|
| 1     | python -m venv mv              | Cria a máquina virtual Python dentro da pasta do projeto. Execute uma única vez por projeto. |
| 2     | mv\Scripts\Activate.ps1        | Ativa a máquina virtual no PowerShell. Execute toda vez que abrir o projeto.                 |
| 3     | mv\Scripts\Activate.bat        | Ativa a máquina virtual no CMD. Use este no Prompt de Comando, não no PowerShell.            |
| 4     | deactivate                     | Sai da máquina virtual ativa.                                                                |
| 5     | Remove-Item -Recurse -Force mv | Remove a pasta da máquina virtual. Use para recriar o ambiente do zero.                      |

**PowerShell bloqueado:** Se o PowerShell bloquear a ativação por política de execução, use temporariamente:  
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

### 3. Atualizar ferramentas básicas do Python

| Ordem | Comando                                                           | Para que serve                                                                        |
|-------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 1     | <code>python -m pip install --upgrade pip setuptools wheel</code> | Atualiza ferramentas usadas para instalar pacotes Python e reduz erros de instalação. |

### 4. Instalar plugins e dependências

**Boa prática:** Prefira `python -m pip install` em vez de apenas `pip install`, porque isso garante que a instalação será feita no Python do ambiente virtual ativo.

| Ordem | Comando                                                         | Para que serve                                                                              |
|-------|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| 1     | <code>python -m pip install pytest</code>                       | Instala o Pytest, framework usado para executar testes automatizados.                       |
| 2     | <code>python -m pip install selenium</code>                     | Instala o Selenium. Use apenas se o projeto também tiver testes Selenium.                   |
| 3     | <code>python -m pip install pytest-playwright playwright</code> | Instala o Playwright e sua integração com Pytest.                                           |
| 4     | <code>playwright install</code>                                 | Baixa os navegadores usados pelo Playwright, como Chromium, Firefox e WebKit.               |
| 5     | <code>python -m pip install allure-pytest</code>                | Instala o plugin que gera resultados compatíveis com relatórios Allure.                     |
| 6     | <code>python -m pip install pytest-order</code>                 | Permite usar <code>@pytest.mark.order(1)</code> , <code>@pytest.mark.order(2)</code> , etc. |
| 7     | <code>python -m pip install pytest-base-url</code>              | Adiciona suporte a <code>base_url</code> no Pytest, caso o projeto use esse recurso.        |

### Instalação completa em um único comando

```
python -m pip install pytest pytest-playwright playwright allure-pytest pytest-order pytest-base-url selenium
```

### 5. Arquivo requirements.txt

- O nome correto do arquivo é `requirements.txt`.
- Ele guarda as dependências do projeto para facilitar instalação em outra máquina ou novo ambiente virtual.
- Evite salvar a pasta `mv` no Git. Normalmente ela deve ficar no `.gitignore`.

| Ordem | Comando                                                 | Para que serve                                                                                              |
|-------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1     | <code>python -m pip freeze &gt; requirements.txt</code> | Gera ou atualiza o arquivo <code>requirements.txt</code> com as bibliotecas instaladas no ambiente virtual. |
| 2     | <code>python -m pip install -r requirements.txt</code>  | Instala todas as dependências listadas no <code>requirements.txt</code> .                                   |

## 6. Estrutura recomendada do projeto

```
nome_projeto/  
  pages/  
    __init__.py  
    cadastrar_produto_po.py  
  resources/  
    __init__.py  
  tests/  
    __init__.py  
    test_cadastrar_produto.py  
  mv/  
    ambiente virtual Python  
  pytest.ini  
  requirements.txt
```

**Importante:** Os arquivos `__init__.py` ajudam o Python a reconhecer as pastas como pacotes e evitam problemas de importação em alguns projetos.

## 7. Configuração do `pytest.ini`

```
[pytest]  
addopts = -v -s --headed --browser firefox --alluredir=allure-results --clean-alluredir  
testpaths = tests  
markers =  
    order: define a ordem de execução dos testes
```

| Opção                                   | Resumo                                                                                    |
|-----------------------------------------|-------------------------------------------------------------------------------------------|
| <code>-v</code>                         | Mostra o resultado dos testes com mais detalhes.                                          |
| <code>-s</code>                         | Permite visualizar prints e mensagens no terminal.                                        |
| <code>--headed</code>                   | Executa o navegador visível. Se o browser já abre, esse item pode estar configurado aqui. |
| <code>--browser firefox</code>          | Define Firefox como navegador padrão. Também pode usar chromium ou webkit.                |
| <code>--alluredir=allure-results</code> | Define a pasta onde os resultados do Allure serão salvos.                                 |
| <code>--clean-alluredir</code>          | Limpa os resultados antigos antes de uma nova execução.                                   |
| <code>testpaths = tests</code>          | Define a pasta padrão onde o Pytest deve procurar os testes.                              |
| <code>markers = order</code>            | Registra o marcador order para evitar warnings do Pytest.                                 |

## 8. Ordenar execução dos testes

| Ordem | Comando                                         | Para que serve                                              |
|-------|-------------------------------------------------|-------------------------------------------------------------|
| 1     | <code>python -m pip install pytest-order</code> | Instala o plugin responsável pela ordenação dos testes.     |
| 2     | <code>@pytest.mark.order(1)</code>              | Coloque acima do teste que deve executar em primeiro lugar. |
| 3     | <code>@pytest.mark.order(2)</code>              | Coloque acima do teste que deve executar em segundo lugar.  |

```
import pytest

@pytest.mark.order(1)
def test_abrir_cadastro_produto():
    pass

@pytest.mark.order(2)
def test_fazer_login():
    pass
```

## 9. Comandos para rodar o projeto

| Ordem | Comando                                                                 | Para que serve                                                                                          |
|-------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 1     | <code>python -m pytest</code>                                           | Executa todos os testes encontrados na pasta definida em testpaths.                                     |
| 2     | <code>python -m pytest tests\nome_arquivo.py</code>                     | Executa apenas um arquivo de teste específico.                                                          |
| 3     | <code>python -m pytest -v tests\nome_arquivo.py</code>                  | Executa um arquivo específico com saída detalhada. Se -v já estiver no pytest.ini, não precisa repetir. |
| 4     | <code>python -m pytest tests\nome_arquivo.py::test_nome_do_teste</code> | Executa somente um teste específico dentro de um arquivo.                                               |
| 5     | <code>python -m pytest --headed</code>                                  | Força o navegador a abrir visivelmente, caso isso não esteja no pytest.ini.                             |
| 6     | <code>python -m pytest --browser=chromium</code>                        | Executa os testes usando Chromium.                                                                      |
| 7     | <code>python -m pytest --browser=firefox</code>                         | Executa os testes usando Firefox.                                                                       |
| 8     | <code>python -m pytest --browser=webkit</code>                          | Executa os testes usando WebKit.                                                                        |

## 10. Comandos do Allure

**Pré-requisito:** O pacote allure-pytest gera os resultados, mas para usar o comando allure no terminal você também precisa instalar o Allure Commandline no Windows e deixar o executável no PATH.

| Ordem | Comando                                                                                | Para que serve                                                        |
|-------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 1     | <code>python -m pytest --alluredir=reports</code>                                      | Executa os testes e salva os resultados brutos na pasta reports.      |
| 2     | <code>python -m pytest -v tests\test_pericia_medica_test.py --alluredir reports</code> | Executa um arquivo específico e salva os resultados na pasta reports. |
| 3     | <code>allure serve reports</code>                                                      | Gera e abre temporariamente o relatório HTML no navegador.            |
| 4     | <code>allure generate reports -o allure-report --clean</code>                          | Gera uma pasta organizada com o relatório final em allure-report.     |
| 5     | <code>allure open allure-report</code>                                                 | Abre o relatório Allure já gerado.                                    |
| 6     | <code>allure generate allure-results -o allure-report --clean</code>                   | Gera relatório final a partir da pasta allure-results.                |
| 7     | <code>allure serve allure-results</code>                                               | Gera e abre relatório temporário usando allure-results.               |

## 11. Sequências prontas para uso

### 11.1 Primeira configuração do projeto

```
cd "C:\Users\RODRIGO COSTA\Desktop\Automação\Playwright\Automacao"
python --version
python -m venv mv
mv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
python -m pip install pytest pytest-playwright playwright allure-pytest pytest-order pytest-base-url
selenium
playwright install
python -m pip freeze > requirements.txt
python -m pytest
```

### 11.2 Execução diária

```
cd "C:\Users\RODRIGO COSTA\Desktop\Automação\Playwright\Automacao"
mv\Scripts\Activate.ps1
python -m pytest
```

### 11.3 Recriar o ambiente virtual do zero

```
deactivate
Remove-Item -Recurse -Force mv
python -m venv mv
.\mv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r requirements.txt
playwright install
python -m pytest
```

## 12. Seletores no Playwright

| Exemplo                                  | Tipo correto           | Uso                                                                |
|------------------------------------------|------------------------|--------------------------------------------------------------------|
| page.locator("#login")                   | CSS por ID             | Seleciona elemento com id="login".                                 |
| page.locator(".btn-primary")             | CSS por classe         | Seleciona elemento com class="btn-primary".                        |
| page.locator('[name="q"]')               | CSS por atributo       | Seleciona elemento com atributo name="q".                          |
| page.locator("input[name='q']")          | CSS por tag e atributo | Seleciona input com name="q". Observação: isso não é XPath, é CSS. |
| page.locator("//input[@name='q']")       | XPath                  | Seleciona input com name="q" usando XPath.                         |
| page.locator("//h1[text()='TextoAqui']") | XPath                  | Seleciona h1 cujo texto seja TextoAqui.                            |

## 13. Observações importantes

- Se o pytest.ini já possui -v no addopts, não é necessário rodar python -m pytest -v. Use apenas python -m pytest.
- Se o pytest.ini já possui --headed, o navegador abrirá mesmo sem informar --headed no terminal.
- Se aparecer warning de pytest.mark.order, instale pytest-order e registre o marker no pytest.ini.
- No Playwright Python, o import correto é from playwright.sync\_api import Page, expect.

- Evite instalar pacotes antigos como pytest-allure-adaptor ou allure sem necessidade. Prefira allure-pytest.
- Depois de criar ou recriar a máquina virtual, sempre rode playwright install para baixar os navegadores.

## 14. Checklist final de instalação

| Item                              | Conferência                                                         |
|-----------------------------------|---------------------------------------------------------------------|
| Python instalado                  | Rodar python --version e confirmar que o comando responde.          |
| Ambiente virtual ativo            | O terminal deve aparecer com (mv) antes do caminho.                 |
| Dependências instaladas           | Rodar python -m pip list ou reinstalar com requirements.txt.        |
| Navegadores Playwright instalados | Rodar playwright install após criar ou recriar o ambiente.          |
| pytest.ini configurado            | Conferir addopts, testpaths e markers.                              |
| Testes executando                 | Rodar python -m pytest e validar se os testes passam.               |
| Allure disponível                 | Rodar allure --version, caso queira abrir relatórios pelo terminal. |